DATA ANALYTICS LAB SHEET 6

Name: ANURAG PARMAR

Roll No.: 8

Section: B

CU ID: CU24250076

B.Tech CSE

Experiment No. 6

Predictive Analytics using Linear Regression and Model Performance Evaluation

AIM

To develop a Linear Regression model using Python for predicting continuous outcomes and evaluate its performance using appropriate regression metrics.

DATASET

Diabetes Dataset available directly through Scikit-learn. It is a suitable regression dataset containing numerical features and a continuous target variable.

PURPOSE

Predictive analytics uses historical data and machine learning algorithms to forecast future outcomes. Linear Regression is a supervised learning algorithm used for predicting continuous numerical values by establishing relationships between independent and dependent variables.

1. Import Required Libraries

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("Libraries imported successfully.")
```

2. Load the Dataset

Code:

```
data = load_diabetes()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

df = X.copy()
df["target"] = y

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

```
        age       sex       bmi        bp        s1        s2        s3  \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142

         s4        s5        s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
```

3. Dataset Inspection and Basic EDA

Code:

```
print("Column names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
display(df.isnull().sum())

print("\nSummary statistics:")
display(df.describe())
```

```
age       0
sex       0
bmi       0
bp        0
s1        0
s2        0
s3        0
s4        0
s5        0
s6        0
target    0
dtype: int64
```

```
                age           sex           bmi            bp            s1  \
count  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02
mean  -2.511817e-19  1.230790e-17 -2.245564e-16 -4.797570e-17 -1.381499e-17
std    4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02
min   -1.072256e-01 -4.464164e-02 -9.027530e-02 -1.123988e-01 -1.267807e-01
25%   -3.729927e-02 -4.464164e-02 -3.422907e-02 -3.665608e-02 -3.424784e-02
50%    5.383060e-03 -4.464164e-02 -7.283766e-03 -5.670422e-03 -4.320866e-03
75%    3.807591e-02  5.068012e-02  3.124802e-02  3.564379e-02  2.835801e-02
max    1.107267e-01  5.068012e-02  1.705552e-01  1.320436e-01  1.539137e-01

                 s2            s3            s4            s5            s6  \
count  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02
mean   3.918434e-17 -5.777179e-18 -9.042540e-18  9.293722e-17  1.130318e-17
std    4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02
min   -1.156131e-01 -1.023071e-01 -7.639450e-02 -1.260971e-01 -1.377672e-01
25%   -3.035840e-02 -3.511716e-02 -3.949338e-02 -3.324559e-02 -3.317903e-02
50%   -3.819065e-03 -6.584468e-03 -2.592262e-03 -1.947171e-03 -1.077698e-03
75%    2.984439e-02  2.931150e-02  3.430886e-02  3.243232e-02  2.791705e-02
max    1.987880e-01  1.811791e-01  1.852344e-01  1.335973e-01  1.356118e-01

           target
count  442.000000
mean   152.133484
std     77.093005
min     25.000000
25%     87.000000
50%    140.500000
75%    211.500000
max    346.000000
```

4. Feature and Target Selection

Code:

```
X = df.drop(columns=["target"])
y = df["target"]

print("Independent variables:")
print(X.columns.tolist())

print("\nDependent variable:")
print("target")

print("\nNumber of features:", X.shape[1])
print("Number of observations:", X.shape[0])
```

5. Split Dataset into Training and Testing Sets

Code:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Train-test ratio: 80:20")
```

6. Train Linear Regression Model

Code:

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
print("Intercept:", model.intercept_)
```

7. Make Predictions

Code:

```
y_pred = model.predict(X_test)

comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

display(comparison.head(10))
```

```
   Actual   Predicted
0   219.0  139.547558
1    70.0  179.517208
2   202.0  134.038756
3   230.0  291.417029
4   111.0  123.789659
5    84.0   92.172347
6   242.0  258.232389
7   272.0  181.337321
8    94.0   90.224113
9    96.0  108.633759
```

8. Actual vs Predicted Values

Code:

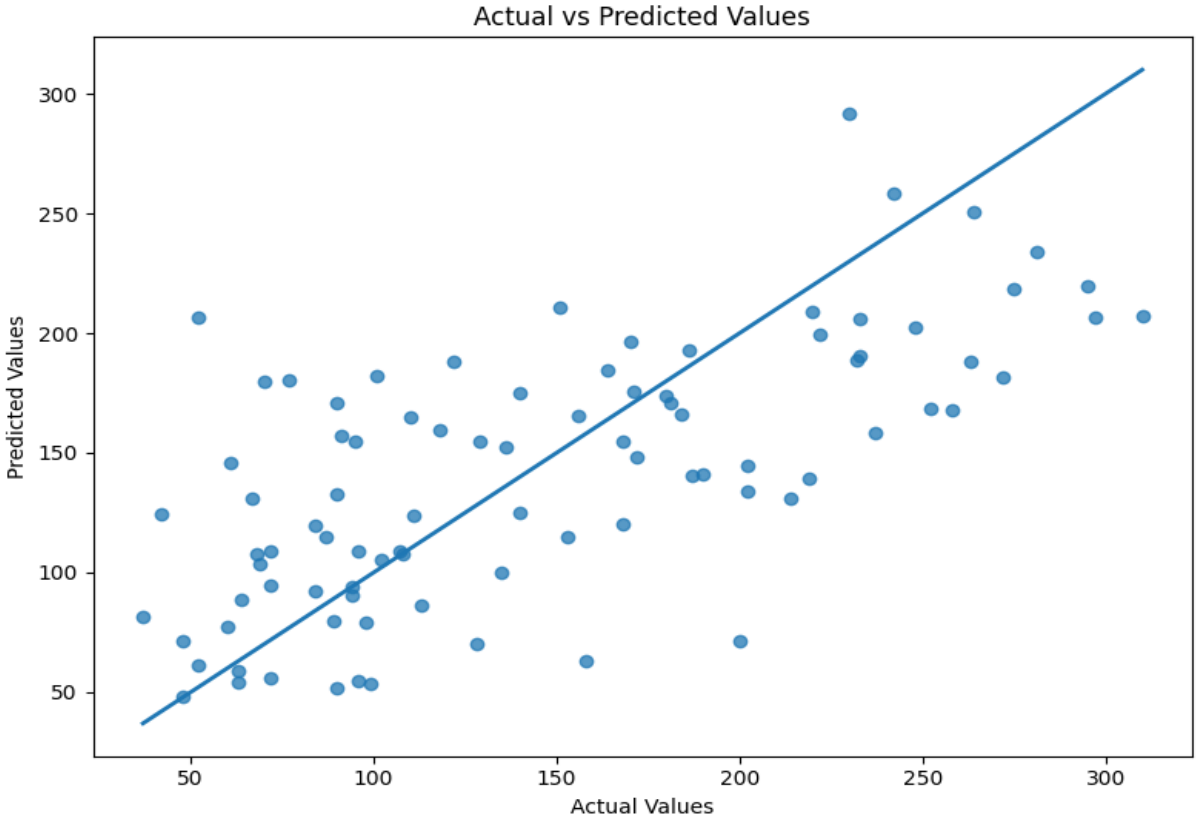
```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.75)

minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot([minimum, maximum], [minimum, maximum], linewidth=2)

plt.title("Actual vs Predicted Values")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.tight_layout()
plt.show()
```

```
<Figure size 800x600 with 1 Axes>
```



9. Regression Line Visualization

Code:

```
# Visualize the relationship between one selected feature and the target.
# The complete model uses all available features.

feature = "bmi"

simple_model = LinearRegression()
simple_model.fit(X_train[[feature]], y_train)

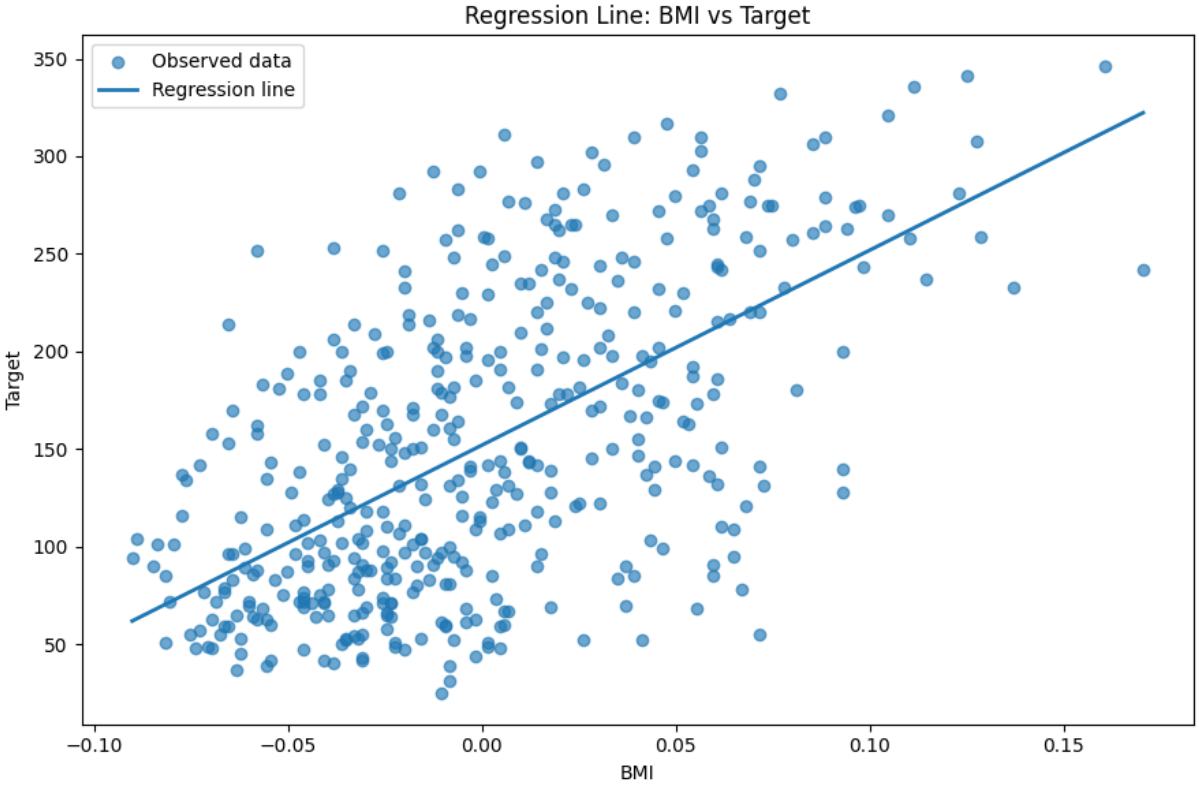
feature_values = np.linspace(
    X[feature].min(),
    X[feature].max(),
    100
).reshape(-1, 1)

line_predictions = simple_model.predict(feature_values)

plt.figure(figsize=(9,6))
plt.scatter(X[feature], y, alpha=0.65, label="Observed data")
plt.plot(feature_values, line_predictions, linewidth=2, label="Regression line")

plt.title("Regression Line: BMI vs Target")
plt.xlabel("BMI")
plt.ylabel("Target")
plt.legend()
plt.tight_layout()
plt.show()
```

```
<Figure size 900x600 with 1 Axes>
```



10. Model Performance Evaluation

Code:

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

metrics = pd.DataFrame({
    "Metric": ["MAE", "MSE", "RMSE", "R² Score"],
    "Value": [mae, mse, rmse, r2]
})

display(metrics)

print(f"MAE  = {mae:.4f}")
print(f"MSE  = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"R²   = {r2:.4f}")
```

```
     Metric        Value
0       MAE    42.794095
1       MSE  2900.193628
2      RMSE    53.853446
3  R² Score     0.452603
```

11. Analyze Regression Coefficients

Code:

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

coefficients["Absolute Impact"] = coefficients["Coefficient"].abs()
coefficients = coefficients.sort_values("Absolute Impact", ascending=False)

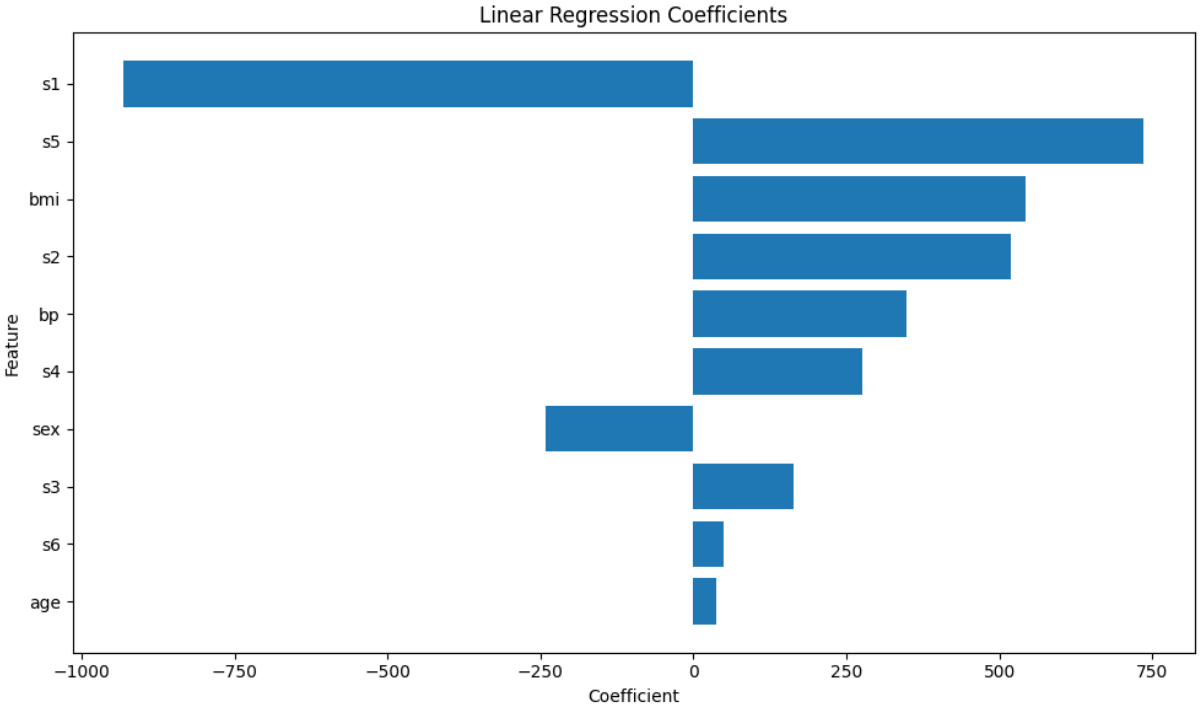
display(coefficients)

plt.figure(figsize=(10,6))
plt.barh(coefficients["Feature"], coefficients["Coefficient"])
plt.title("Linear Regression Coefficients")
plt.xlabel("Coefficient")
plt.ylabel("Feature")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

```
  Feature  Coefficient  Absolute Impact
4      s1  -931.488846       931.488846
```

```
8      s5   736.198859       736.198859
2     bmi   542.428759       542.428759
5      s2   518.062277       518.062277
3      bp   347.703844       347.703844
7      s4   275.317902       275.317902
1     sex  -241.964362       241.964362
6      s3   163.419983       163.419983
9      s6    48.670657        48.670657
0     age    37.904021        37.904021
```

```
<Figure size 1000x600 with 1 Axes>
```



12. Residual Analysis

Code:

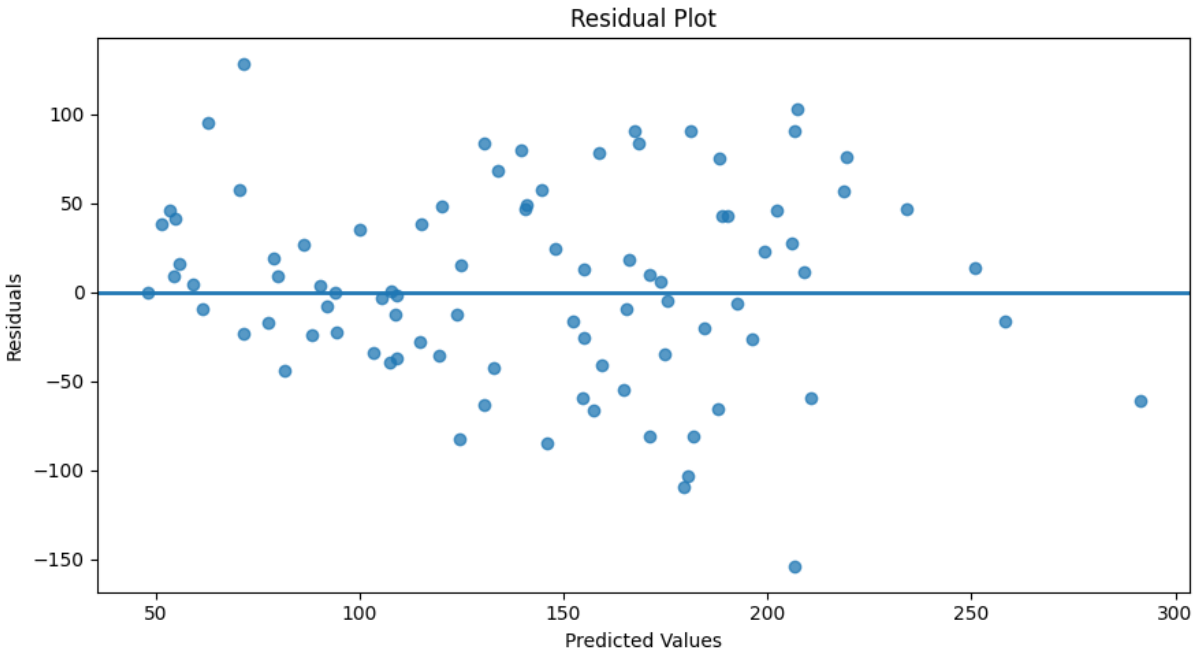
```
residuals = y_test.values - y_pred

plt.figure(figsize=(9,5))
plt.scatter(y_pred, residuals, alpha=0.75)
plt.axhline(0, linewidth=2)

plt.title("Residual Plot")
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.tight_layout()
plt.show()

print("Residual mean:", residuals.mean())
```

```
<Figure size 900x500 with 1 Axes>
```



13. Interpretation of Results

Code:

```
print("MODEL INTERPRETATION")
print("--------------------")
print(f"1. MAE indicates an average absolute prediction error of {mae:.2f}.")
print(f"2. RMSE is {rmse:.2f}; because errors are squared before averaging, it gives greater importance to large errors.")
print(f"3. R² Score is {r2:.4f}, representing the proportion of target variance explained by the model.")
print("4. Regression coefficients indicate the direction and magnitude of the relationship between each feature and the target while c
print("5. The actual-vs-predicted plot helps assess how closely predictions follow observed values.")
print("6. The residual plot helps identify systematic patterns that may indicate model limitations.")
```

RESULT

A Linear Regression model was successfully developed using the Diabetes Dataset. The dataset was inspected and divided into training and testing sets using an 80:20 ratio. The model was trained, predictions were generated, a regression line and actual-vs-predicted visualization were created, and the model was evaluated using MAE, MSE, RMSE, and R² Score. Regression coefficients and residuals were also analyzed.

Answers to Questions

Q1. What is Predictive Analytics, and how is it used in real-world applications?

Predictive Analytics uses historical data, statistical methods, and machine learning algorithms to estimate future or unknown outcomes. It is used in applications such as sales forecasting, demand prediction, financial analysis, risk assessment, and customer behavior prediction.

Q2. Explain the working principle of the Linear Regression algorithm.

Linear Regression models the relationship between independent variables and a continuous dependent variable. It estimates coefficients for the features so that the difference between actual and predicted values is minimized, commonly using the least-squares method.

Q3. Differentiate between dependent and independent variables with suitable examples.

An independent variable is a feature used to make a prediction, while a dependent variable is the outcome being predicted. For example, when predicting house price, house area and number of rooms can be independent variables and house price is the dependent variable.

Q4. Why is it necessary to split the dataset into training and testing sets?

The training set is used to learn the model parameters, while the testing set is used to evaluate the model on data that was not used during training. This provides an indication of how well the model generalizes to unseen data.

Q5. What is the significance of the R² Score in regression analysis?

R^2 Score measures the proportion of variation in the dependent variable that is explained by the regression model. A higher R^2 generally indicates that the model explains more of the observed variation, although it should be interpreted together with other metrics and the problem context.

Q6. Differentiate between MAE, MSE, and RMSE. Which metric is more sensitive to large prediction errors?

MAE is the average absolute prediction error. MSE is the average squared prediction error. RMSE is the square root of MSE and is expressed in the same units as the target. MSE and RMSE are more sensitive to large errors because errors are squared.

Q7. What assumptions should be satisfied before applying Linear Regression?

Important assumptions include a reasonably linear relationship between predictors and the target, independent observations/errors, approximately constant error variance (homoscedasticity), and limited problematic multicollinearity among predictors. Residuals should also be examined for systematic patterns.

Q8. How can overfitting and underfitting affect the performance of a regression model?

Overfitting occurs when a model learns noise or overly specific patterns from training data, resulting in poor generalization. Underfitting occurs when the model is too simple to capture important relationships, causing poor performance on both training and testing data.

Q9. Mention any three real-world applications of Linear Regression in business or industry.

Three applications are:

1. Sales and revenue forecasting.
2. House price prediction.
3. Demand or cost estimation.

Q10. How can feature selection improve the accuracy and interpretability of a predictive model?

Feature selection removes irrelevant or redundant variables. It can reduce noise, simplify the model, reduce multicollinearity, improve generalization in some cases, decrease computational complexity, and make the relationships represented by the model easier to interpret.